DSA0605 – Data Handling and Visualization

CO1-AT2-Data Interpretation

Name: P. Siddardha

Reg.no: 192424271

Question 1: Color Scales for Data Representation [R]

Dataset: Employee Performance Score

Employee	Performance Score
E1	62
E2	78
E3	85
E4	70
E5	91
E6	55

1. Write an R program to create a bar chart using a sequential color scale based on Performance Score.

```
# Question 1.1: Sequential Color Scale

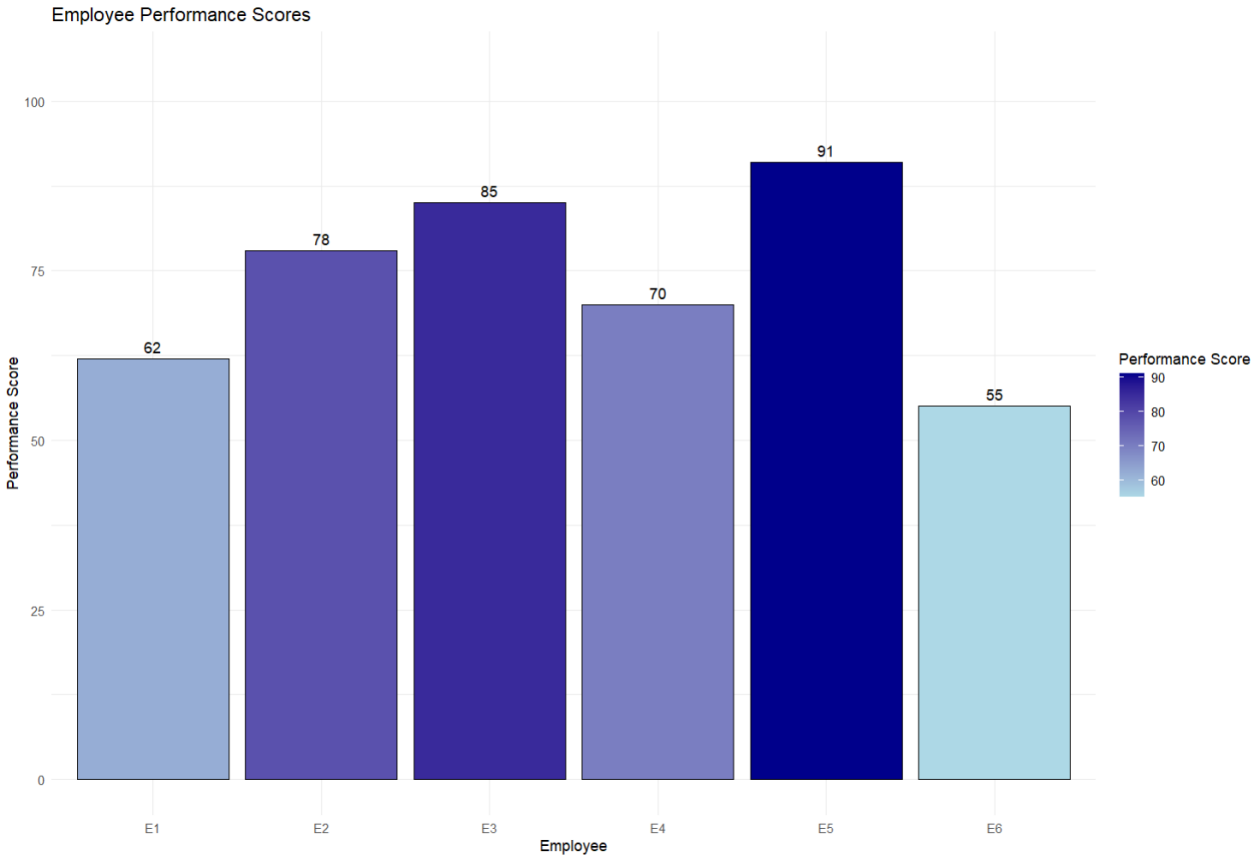
# Install ggplot2 if required
# install.packages("ggplot2")

library(ggplot2)

# Create dataset
employees <- data.frame(
  Employee = c("E1", "E2", "E3", "E4", "E5", "E6"),
  Score = c(62, 78, 85, 70, 91, 55)
)

# Create sequential color bar chart
ggplot(employees, aes(x = Employee, y = Score, fill = Score)) +
  geom_col(color = "black") +
  geom_text(aes(label = Score), vjust = -0.5) +
  scale_fill_gradient(
    low = "lightblue",
    high = "darkblue",
    name = "Performance Score"
  ) +
  labs(
    title = "Employee Performance Scores",
    x = "Employee",
    y = "Performance Score"
  ) +
  coord_cartesian(ylim = c(0, 105)) +
```

theme_minimal()



Explanation:

- ggplot() initializes the visualization using the employee dataset.
- aes() maps Employee to the x-axis, Score to the y-axis, and Score to the fill colour.
- geom_col() creates bars whose heights represent the actual scores.
- geom_text() displays the score above each bar.
- scale_fill_gradient() creates a sequential colour scale from light blue to dark blue.
- labs() adds the title and axis labels.
- theme_minimal() provides a clean appearance.

Expected output: A bar chart with six employee bars, numeric labels, and a colour gradient in which higher scores appear darker.

2. Identify the employees with the highest and lowest performance scores.

- Highest performance: E5, with a score of 91.
- Lowest performance: E6, with a score of 55.

In the visualization, E5 has the tallest, darkest bar, whereas E6 has the shortest, lightest bar.

3. Explain why a sequential color scale is appropriate.

A sequential colour scale is appropriate because performance scores are quantitative values with a meaningful low-to-high order. A gradual change in colour intensity communicates increasing magnitude.

- Light colours represent lower scores.
- Dark colours represent higher scores.
- The gradient makes high and low values easier to identify.
- A single colour family maintains visual consistency.

Therefore, a sequential colour scale effectively represents the differences in employee performance.

4. Suggest a color palette suitable for color-blind users.

The Viridis palette is a suitable option because it is designed to remain perceptually distinguishable for many users with colour-vision deficiencies.

Replace the scale_fill_gradient() line with:

scale_fill_viridis_c(option = "C", name = "Performance Score")

This requires the viridis package:

install.packages("viridis")

Viridis provides a perceptually ordered gradient. Numeric labels should also be retained so that the values can be interpreted without relying only on colour.

5. Explain the difference between sequential and categorical color scales.

Feature	Sequential colour scale	Categorical colour scale
Purpose	Represents ordered numerical values	Distinguishes separate groups
Data type	Quantitative	Nominal
Colour pattern	Gradual progression in intensity	Distinct colours
Example	Employee performance scores	Employee departments
Suitable palette	Blues, Viridis, Cividis	Set2, Dark2

Conclusion: Sequential scales represent magnitude and order, while categorical scales distinguish groups without implying a numerical ranking.

Question 2: Coordinate Systems (Cartesian and Polar) [R]

Dataset: Hour of Day and Store Footfall

Hour of Day	Store Footfall
6 AM	10
9 AM	45
12 PM	80
3 PM	60
6 PM	95

9 PM	70
12 AM	20
3 AM	5

1. Write an R program to visualize the dataset using a Cartesian coordinate system (bar chart).

```
# Question 2.1: Cartesian Bar Chart

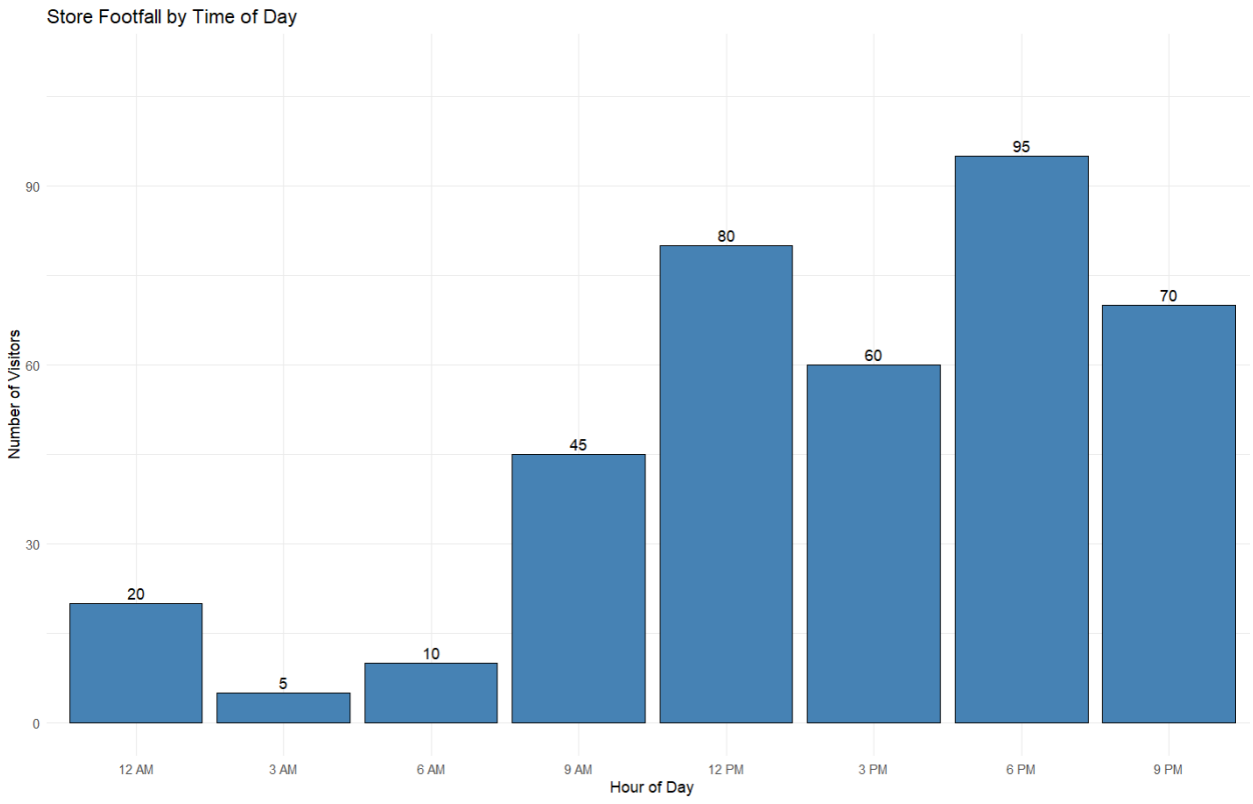
library(ggplot2)

# Create dataset in chronological order
footfall <- data.frame(
  Hour = c("12 AM", "3 AM", "6 AM", "9 AM",
           "12 PM", "3 PM", "6 PM", "9 PM"),
  Visitors = c(20, 5, 10, 45, 80, 60, 95, 70)
)

footfall$Hour <- factor(
  footfall$Hour,
  levels = footfall$Hour
)

# Create Cartesian bar chart
ggplot(footfall, aes(x = Hour, y = Visitors)) +
  geom_col(fill = "steelblue", color = "black") +
  geom_text(aes(label = Visitors), vjust = -0.4) +
  labs(
    title = "Store Footfall by Time of Day",
    x = "Hour of Day",
    y = "Number of Visitors"
  ) +
  coord_cartesian(ylim = c(0, 110)) +
  theme_minimal()
```

Explanation: The Cartesian coordinate system uses the horizontal axis to represent time and the vertical axis to represent the number of visitors. The bar heights make comparisons between different times straightforward.



2. Write another R program using coord_polar() to visualize the same data on a 24-hour clock-style polar coordinate system.

```
# Question 2.2: Polar Coordinate Bar Chart

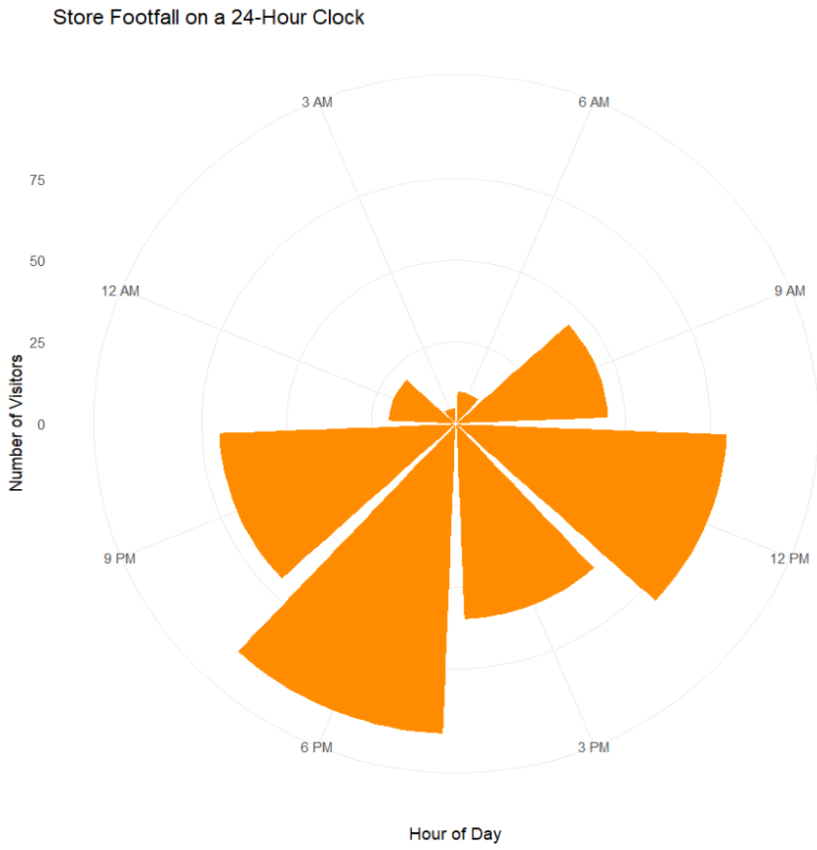
library(ggplot2)

# Create dataset
footfall <- data.frame(
  Hour = c("12 AM", "3 AM", "6 AM", "9 AM",
           "12 PM", "3 PM", "6 PM", "9 PM"),
  Visitors = c(20, 5, 10, 45, 80, 60, 95, 70)
)

footfall$Hour <- factor(
  footfall$Hour,
  levels = footfall$Hour
)

# Create polar bar chart
```

```
ggplot(footfall, aes(x = Hour, y = Visitors)) +
  geom_col(fill = "darkorange", color = "white") +
  coord_polar(theta = "x", start = -pi / 2,
              direction = 1) +
  labs(
    title = "Store Footfall on a 24-Hour Clock",
    x = "Hour of Day",
    y = "Number of Visitors"
  ) +
  theme_minimal()
```



Explanation:

- coord_polar() converts the Cartesian chart into a circular representation.
- theta = "x" maps the time categories around the circle.
- start = -pi/2 rotates the starting position.
- direction = 1 sets the direction in which the categories progress.
- The chronological factor levels arrange the observations in clock order.

The polar chart represents the eight sampled times around a circle. Since observations are available at three-hour intervals, it does not display every hour individually.

3. Compare the effectiveness of both visualizations.

Feature	Cartesian system	Polar system
Representation	Rectangular axes	Circular arrangement
Value comparison	Easy and accurate	Comparatively difficult
Time sequence	Ordered categories	Cyclic pattern
Identifying maximum	Easy using bar height	Possible using radial length
Main advantage	Clear numerical comparison	Emphasizes the daily cycle
Limitation	Less emphasis on repetition	Radial comparisons may be harder

The Cartesian chart is more effective for comparing exact footfall values. The polar chart is more effective for communicating the circular nature of time.

4. Explain which coordinate system is more appropriate for cyclical/time-of-day data.

The polar coordinate system is appropriate when the objective is to emphasize cyclical patterns in time-of-day data. A day repeats after 24 hours, so a circular display can communicate this repeating structure naturally.

However, when precise comparison of visitor counts is the primary objective, a Cartesian chart is generally easier to interpret.

5. Justify your answer based on data interpretation.

The highest footfall is 95 visitors at 6 PM, while the lowest footfall is 5 visitors at 3 AM.

- The Cartesian chart makes the maximum and minimum easy to identify from bar heights.
- The polar chart places the observations in a circular time sequence, helping viewers recognize daytime and nighttime patterns.
- The relatively high evening footfall and low early-morning footfall suggest that store activity varies throughout the day.

Conclusion: Polar coordinates are useful for emphasizing the cyclical structure of time, whereas Cartesian coordinates provide more accurate comparisons of individual values. For this dataset, the Cartesian chart is the better primary chart for comparing footfall.

Question 3: Mapping Data onto Aesthetics [R]

Dataset: Orders, Rating, and Category

Item	Orders	Rating (out of 5)	Category
I1	320	4.2	Electronics
I2	540	3.8	Grocery
I3	210	4.6	Fashion
I4	690	4.0	Grocery
I5	150	4.8	Fashion
I6	410	3.5	Electronics

1. Write an R program using ggplot2 to create a scatter plot with Orders on the x-axis and Rating on the y-axis.
2. Map Category using colour and Orders using another suitable aesthetic, such as size.

```
# Question 3.1 and 3.2: Mapping Data onto Aesthetics

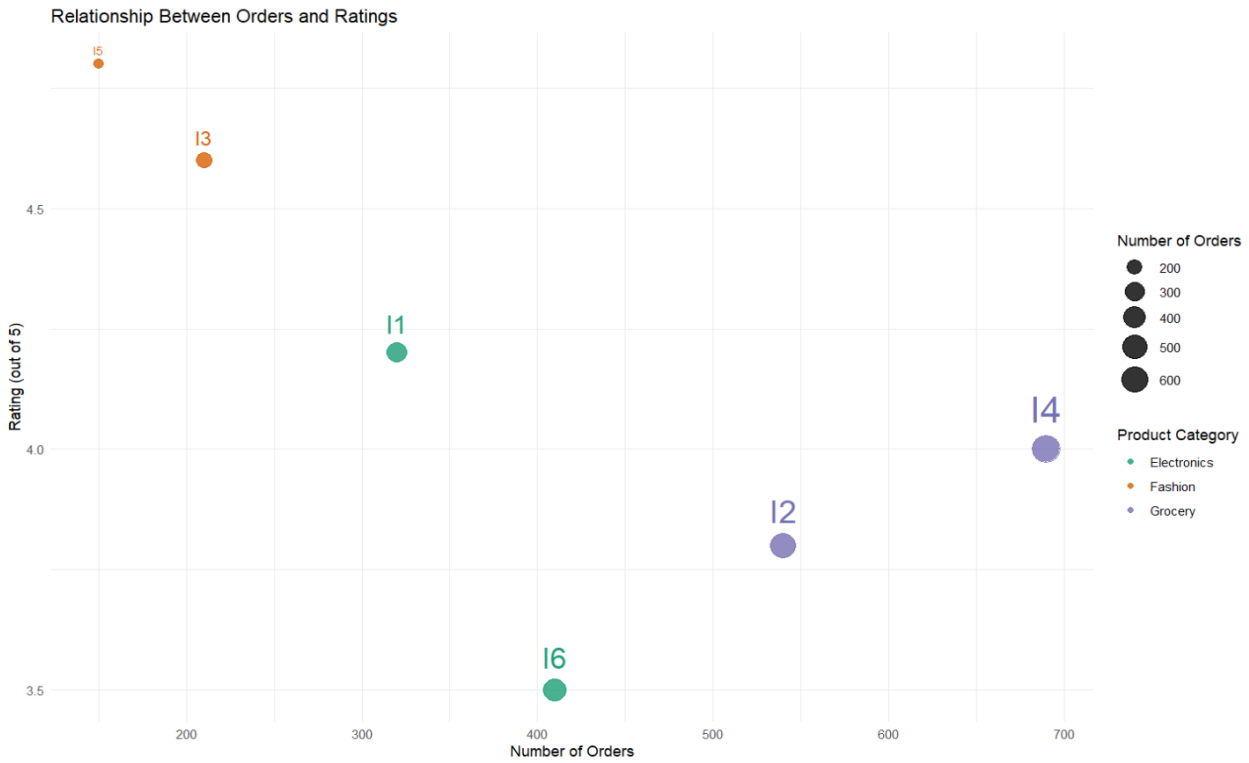
library(ggplot2)

# Create dataset
items <- data.frame(
  Item = c("I1", "I2", "I3", "I4", "I5", "I6"),
  Orders = c(320, 540, 210, 690, 150, 410),
  Rating = c(4.2, 3.8, 4.6, 4.0, 4.8, 3.5),
  Category = c("Electronics", "Grocery", "Fashion",
               "Grocery", "Fashion", "Electronics")
)

# Create scatter plot
ggplot(items, aes(x = Orders, y = Rating,
                  colour = Category, size = Orders)) +
  geom_point(alpha = 0.8) +
  geom_text(aes(label = Item), vjust = -1,
            show.legend = FALSE) +
  scale_colour_brewer(palette = "Dark2") +
  scale_size_continuous(range = c(3, 9)) +
  labs(
    title = "Relationship Between Orders and Ratings",
    x = "Number of Orders",
    y = "Rating (out of 5)",
    colour = "Product Category",
    size = "Number of Orders"
  ) +
  theme_minimal()
```

Explanation:

- geom_point() creates the scatter plot.
- x = Orders maps order counts to horizontal position.
- y = Rating maps ratings to vertical position.
- colour = Category distinguishes product categories.
- size = Orders represents order counts through point size.
- geom_text() labels the products I1 to I6.



Orders is represented by both position and size. This satisfies the question's request for two aesthetic mappings, although the size mapping is redundant because order counts are already represented on the x-axis.

3. Interpret the relationship between Orders and Rating.

The dataset does not show a clear, strong positive relationship between order count and rating.

- I4: Highest order count of 690, with a rating of 4.0.
- I5: Lowest order count of 150, with a rating of 4.8.
- I3: A rating of 4.6 with only 210 orders.
- I6: A rating of 3.5 with 410 orders.

These observations indicate that a product with fewer orders can still receive a high rating. Popularity and customer satisfaction are not necessarily the same.

Because the dataset contains only six observations, a strong general conclusion about the relationship cannot be established from this visualization alone.

4. Identify the type of scale used for each variable.

Variable	Scale type	Explanation
Item	Nominal	Identifies products using labels
Orders	Quantitative, ratio	Counts have a meaningful zero
Rating	Quantitative, usually interval-like for plotting	Numerical ratings can be compared by value and difference
Category	Nominal	Categories have no inherent order

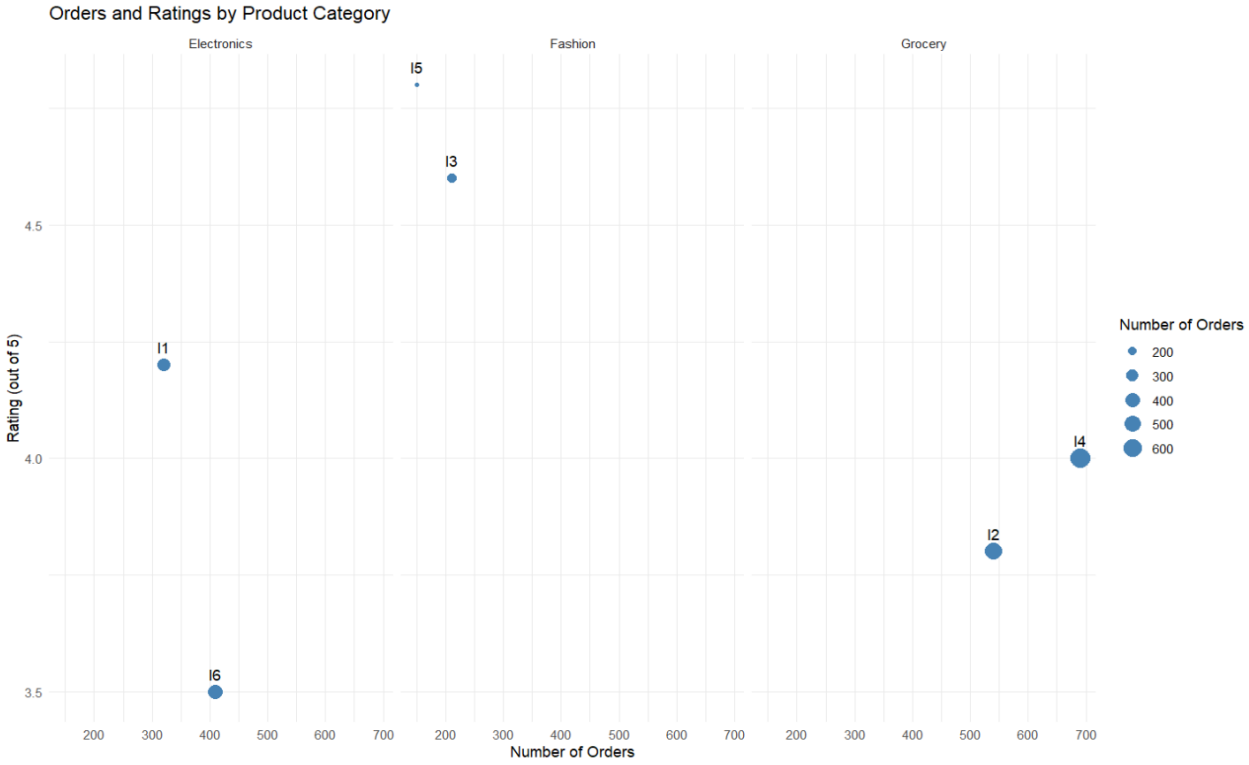
Note: Ratings can also be treated as ordinal if they represent ordered satisfaction levels and equal intervals cannot be assumed. In this scatter plot, ratings are treated as numerical values.

5. Suggest one alternative visualization that better represents the same data.
A faceted scatter plot is an alternative that separates the products by category.

```
# Question 3.5: Alternative Faceted Scatter Plot

library(ggplot2)

ggplot(items, aes(x = Orders, y = Rating)) +
  geom_point(aes(size = Orders), colour = "steelblue") +
  geom_text(aes(label = Item), vjust = -1) +
  facet_wrap(~ Category) +
  labs(
    title = "Orders and Ratings by Product Category",
    x = "Number of Orders",
    y = "Rating (out of 5)",
    size = "Number of Orders"
  ) +
  theme_minimal()
```



Justification: Faceting separates Electronics, Grocery, and Fashion into different panels, reducing visual clutter and making comparisons within each category easier. Common axis scales also help readers compare values across categories.
Conclusion: Mapping data onto position, colour, and size communicates multiple variables in a single visualization. Faceting provides an alternative when separate category-level comparisons are needed.

Question 4: Faceting / Small Multiples [R]

Dataset: Subject, Semester, and Average Score

Subject	Semester	Average Score
Maths	Sem1	72
Maths	Sem2	78
Maths	Sem3	81
Physics	Sem1	65
Physics	Sem2	70
Physics	Sem3	75
English	Sem1	80
English	Sem2	82
English	Sem3	85
CS	Sem1	74
CS	Sem2	83
CS	Sem3	90

1. Write an R program to create a faceted plot showing the average-score trend for each subject separately.

```
# Question 4.1: Faceting / Small Multiples

library(ggplot2)

# Create dataset
scores <- data.frame(
  Subject = c(
    rep("Maths", 3),
    rep("Physics", 3),
    rep("English", 3),
    rep("CS", 3)
  ),
  Semester = rep(c("Sem1", "Sem2", "Sem3"), 4),
  AverageScore = c(
    72, 78, 81,
```

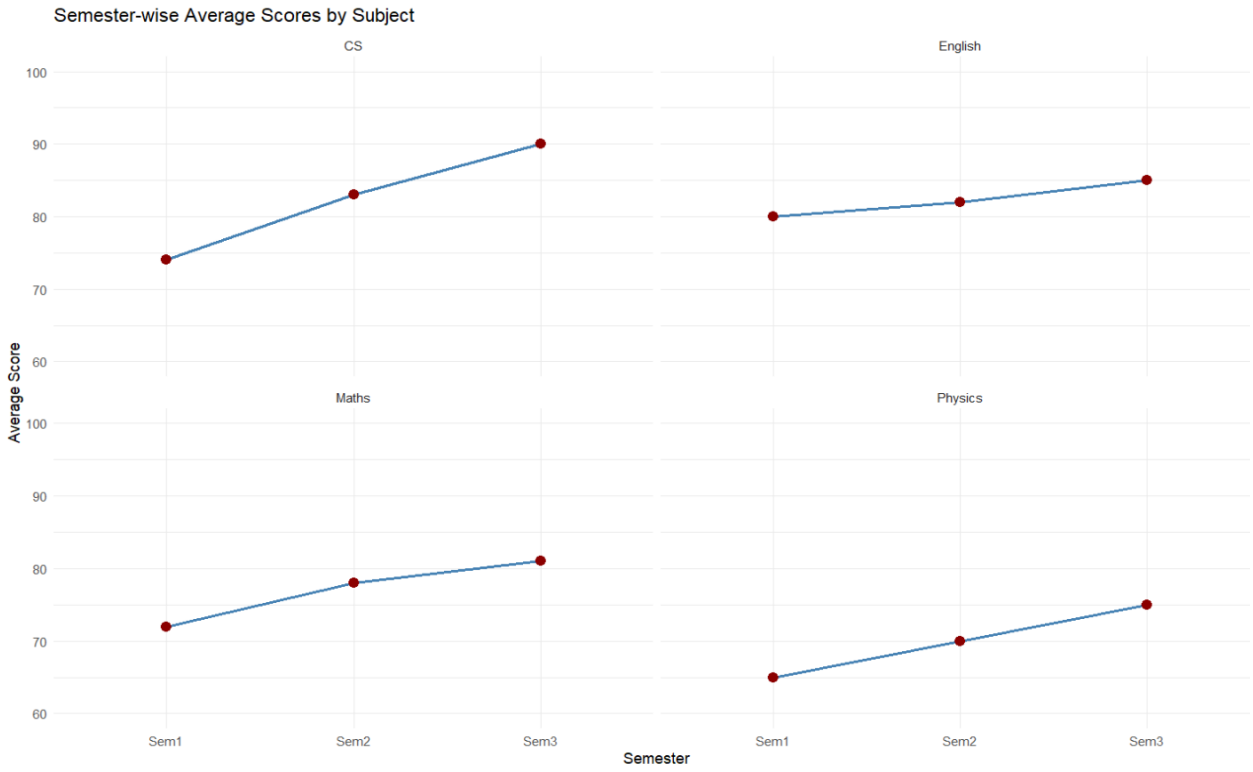
```
    65, 70, 75,
    80, 82, 85,
    74, 83, 90
  )
)

# Arrange semesters in order
scores$Semester <- factor(
  scores$Semester,
  levels = c("Sem1", "Sem2", "Sem3"),
  ordered = TRUE
)

# Create faceted line plot
ggplot(scores, aes(x = Semester, y = AverageScore,
                   group = Subject)) +
  geom_line(colour = "steelblue", linewidth = 1) +
  geom_point(colour = "darkred", size = 3) +
  facet_wrap(~ Subject, ncol = 2) +
  labs(
    title = "Semester-wise Average Scores by Subject",
    x = "Semester",
    y = "Average Score"
  ) +
  coord_cartesian(ylim = c(60, 100)) +
  theme_minimal()
```

Explanation:

- geom_line() displays the change in scores across semesters.
- geom_point() marks individual semester scores.
- facet_wrap(~ Subject) creates a separate panel for each subject.
- ncol = 2 arranges the panels into two columns.
- coord_cartesian() sets a common visible score range for comparing subjects.
- theme_minimal() improves readability.



2. Choose an appropriate variable to facet by and justify the choice.

The appropriate variable is Subject.

Each subject has three semester scores, so creating a separate panel for Maths, Physics, English, and CS makes each subject's trend easier to understand. Semester remains on the x-axis to show the progression over time.

Faceting by Subject reduces overlapping lines and allows readers to compare changes across the four subjects.

3. Compare trends across the facets.

Subject	Sem1	Sem2	Sem3	Improvement
Maths	72	78	81	+9
Physics	65	70	75	+10
English	80	82	85	+5
CS	74	83	90	+16

Observations:

- CS: Shows the greatest improvement, from 74 to 90, increasing by 16 points.
- Physics: Improves steadily from 65 to 75, increasing by 10 points.
- Maths: Increases from 72 to 81, gaining 9 points.
- English: Has the highest initial score of 80 and reaches 85, gaining 5 points.

All four subjects show an upward trend. CS has the largest improvement, while English maintains comparatively high scores across all semesters.

4. Explain the advantage of faceting over a single combined plot.

Faceting divides a complex visualization into multiple smaller plots using the same overall structure.

Its advantages are:

1. Reduced clutter: Each subject appears in a separate panel.
2. Clear comparison: Individual subject trends can be examined easily.
3. Improved readability: Lines from different subjects do not overlap.
4. Consistent scales: Common axis limits support fair comparisons.
5. Better interpretation: Readers can identify improvements and differences across subjects.

For this dataset, faceting provides a clearer view of each subject's progress than a single combined plot with four lines.

5. Suggest a scenario where faceting would not be suitable.

Faceting is not suitable when a dataset contains a very large number of categories, such as 200 subjects or hundreds of departments.

Creating a separate panel for each category would produce too many small plots, making labels and trends difficult to read. A heatmap, grouped visualization, or interactive chart with filtering would be more appropriate.

Conclusion: Faceting is effective when the number of categories is manageable and each category needs an independent view. It becomes less effective when the number of panels is too large.

Question 5: Legends and Annotations [R]

Dataset: Monthly Stock Prices

All prices are in Indian rupees (₹).

Month	Company X (₹)	Company Y (₹)	Company Z (₹)
Jan	410	380	450
Feb	430	395	460
Mar	460	410	440
Apr	480	430	470
May	520	460	510
Jun	500	490	530

1. Write an R program to plot the monthly stock-price trend for all three companies on a single chart with a clear legend.

2. Annotate the highest price for each company using geom_text().

```
# Question 5.1 and 5.2: Legends and Annotations

library(ggplot2)
```

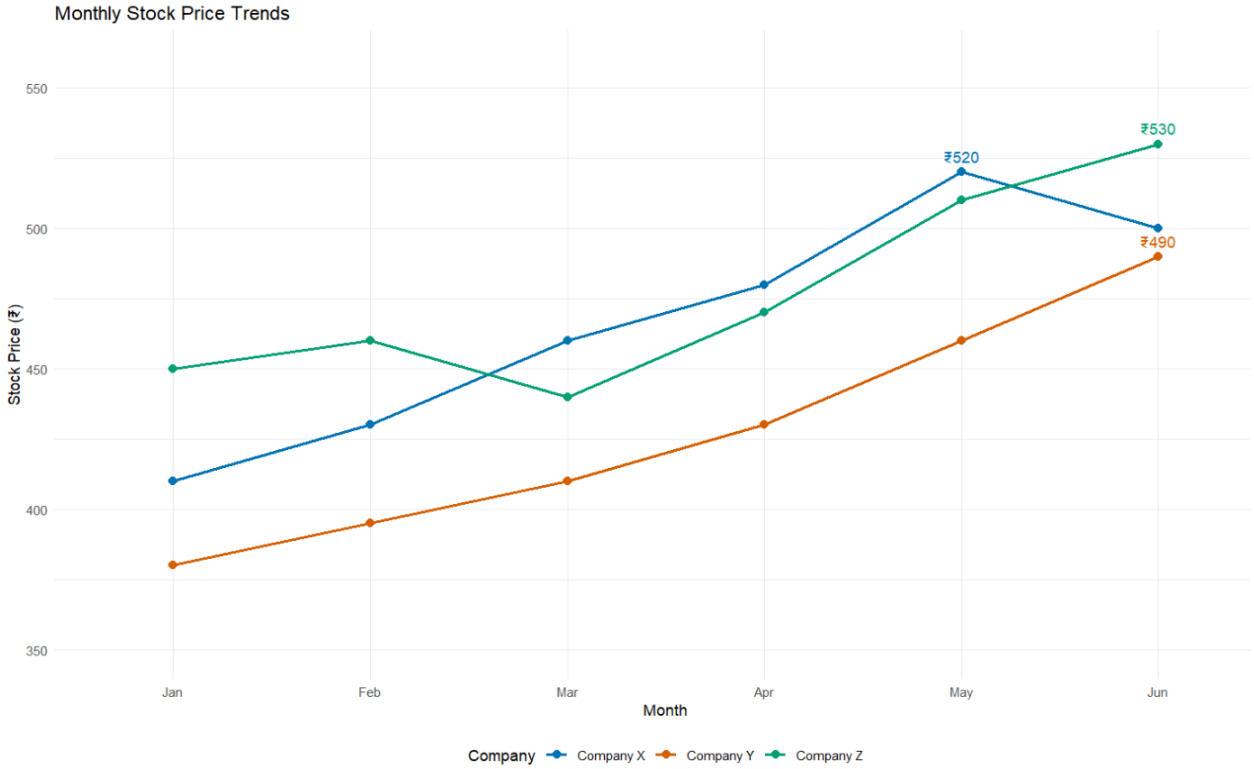
```
# Create dataset
stocks <- data.frame(
  Month = rep(c("Jan", "Feb", "Mar", "Apr", "May", "Jun"), 3),
  Company = rep(c("Company X", "Company Y", "Company Z"),
                each = 6),
  Price = c(
    410, 430, 460, 480, 520, 500,
    380, 395, 410, 430, 460, 490,
    450, 460, 440, 470, 510, 530
  )
)

# Arrange months chronologically
stocks$Month <- factor(
  stocks$Month,
  levels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun")
)

# Find the maximum price for each company
max_points <- stocks[
  stocks$Price == ave(
    stocks$Price,
    stocks$Company,
    FUN = max
  ),
]

# Create line chart
ggplot(stocks, aes(x = Month, y = Price,
                   colour = Company, group = Company)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  geom_text(
    data = max_points,
    aes(label = paste0("₹", Price)),
    vjust = -0.8,
    show.legend = FALSE
  ) +
  scale_colour_manual(
    values = c(
```

```
    "Company X" = "#0072B2",
    "Company Y" = "#D55E00",
    "Company Z" = "#009E73"
  )
) +
labs(
  title = "Monthly Stock Price Trends",
  x = "Month",
  y = "Stock Price (₹)",
  colour = "Company"
) +
coord_cartesian(ylim = c(350, 560)) +
theme_minimal() +
theme(legend.position = "bottom")
```



Explanation:

- geom_line() connects monthly stock prices for each company.
- geom_point() displays the price at each month.
- colour = Company distinguishes the three companies.
- group = Company ensures each company has its own line.
- ave() calculates the maximum price within each company.
- geom_text() labels the maximum-price points.
- scale_colour_manual() assigns distinct colours.

- theme(legend.position = "bottom") places the legend below the chart.

Expected maximum-price annotations:

Company	Highest price	Month
Company X	₹520	May
Company Y	₹490	June
Company Z	₹530	June

These three points should be labelled in the line chart.

3. Explain the role of legends in multi-series visualizations.

A legend explains how visual properties, such as colours or line styles, correspond to different data series.

In the stock-price chart:

- It identifies the line representing each company.
- It helps readers follow each company's trend over six months.
- It prevents confusion when several lines are displayed together.
- It supports comparison of the companies' price movements.

Without a legend or direct labels, readers may struggle to identify which line belongs to which company.

4. Discuss best practices for placing legends to avoid obscuring data.

The following practices improve the readability of multi-series charts:

1. Place the legend outside the main plotting area when possible.
2. Position it at the bottom or right side, depending on available space.
3. Use clear and meaningful category names.
4. Select visually distinguishable colours and line styles.
5. Avoid placing legends over important points or annotations.
6. Keep the legend close enough to the chart for easy reference.
7. Maintain consistent colours across related visualizations.

For this dataset, positioning the legend at the bottom is appropriate because it avoids covering the stock-price lines.

5. Suggest an alternative to a legend for distinguishing categories.

Direct labelling is an alternative in which each line is labelled directly on the chart, usually near its final point.

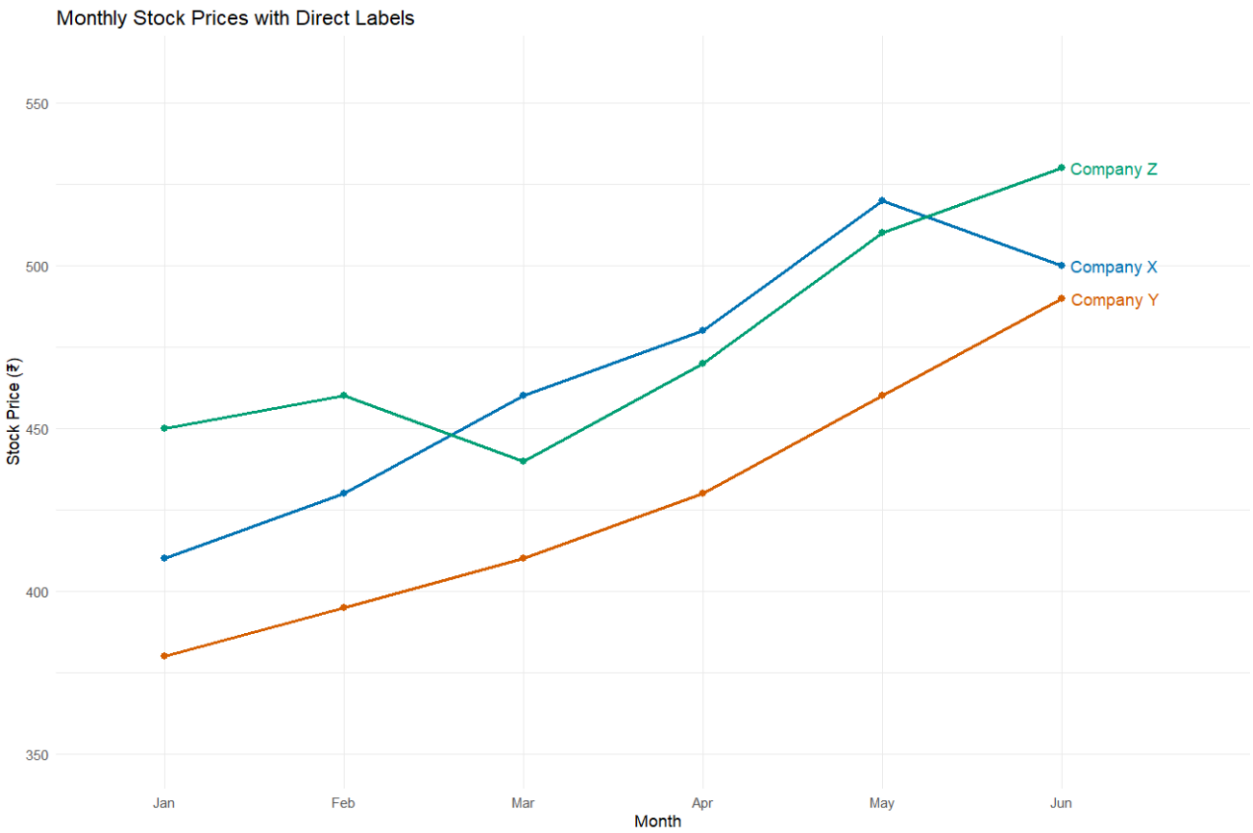
The following R program demonstrates direct labelling.

```
# Question 5.5: Direct Labelling Instead of a Legend

library(ggplot2)
```

```
# Select the June observation for each company
final_points <- stocks[
  stocks$Month == "Jun",
]

# Create directly labelled line chart
ggplot(stocks, aes(x = Month, y = Price,
                   colour = Company, group = Company)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  geom_text(
    data = final_points,
    aes(label = Company),
    hjust = -0.1,
    show.legend = FALSE
  ) +
  scale_colour_manual(
    values = c(
      "Company X" = "#0072B2",
      "Company Y" = "#D55E00",
      "Company Z" = "#009E73"
    )
  ) +
  labs(
    title = "Monthly Stock Prices with Direct Labels",
    x = "Month",
    y = "Stock Price (₹)"
  ) +
  coord_cartesian(
    xlim = c(1, 6.5),
    ylim = c(350, 560),
    clip = "off"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    plot.margin = margin(5.5, 70, 5.5, 5.5)
  )
```



Justification: Direct labelling reduces the need to look back and forth between the chart and the legend. It is especially effective when a visualization contains only a few lines. When there are many categories, a separate legend may be more practical.

Conclusion: Legends provide a key for interpreting multiple data series, while direct labelling can make a small number of lines easier to identify.